CEG5101 Modern Computer Networking

Graduate Assistants: Ms. Poorvi Joshi (e1144005@u.nus.edu), Ms. Le Minh Chau (le_minhchau@u.nus.edu), Ms. Theepana Govintharajah (theepana@u.nus.edu)
Prepared by: Mr. Binghui Wu and Ms. Divya D Kulkarni (Ex - Graduate Assistants)
Lecturer: Assoc. Prof. Mohan Gurusamy
ECE Department, CDE., National University of Singapore

Laboratory 03

Software-Defined Network (SDN) with Ryu Controller

LEARNING OBJECTIVES

Upon successful completion of this laboratory, the students will be able:

i. To become familiar with the concept of SDN and Controller.
ii. To download and install the Ryu controller.
iii. To invoke the Ryu controller as a simple switch using a stock component.
iv. To develop and test the code for a simple switching hub using Ryu's APIs.
v. To learn to test using Curl and HTTP Requests.

EQUIPMENT

i. *Hardware* – Laptop/Computer with internet access
ii. *Software* – Mininet, Linux OS

DISCUSSION

Note: All files required for the exercises are under the provide **ceg_code** folder.

Task 1: To become familiar with the concept of SDN and Controller

1. SDN or software-defined networking is a concept in which application programming interfaces (APIs) are used to enable intelligence within the network.

2. It helps to control the network centrally and consistently regardless of the underlying topology.

3. Centralized intelligence is achieved by separating the control plane and the data plane to get control over the network behaviour.

4. The data plane is the part of the network which handles traffic forwarding and the control plane is the part that instructs or configures the data plane on the traffic forwarding process.

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

5. In a traditional network setup, the control plane and data plane lie together on a single device. However, SDN decouples the control plane from the data plane and implements it as software instead to enable flexibility within a network through programmatic access.

6. In this lab activity, we will learn to use a controller to update data forwarding rules on the Open vSwitch (OVS) to enable communication between different hosts.

Task 2: To download and install Ryu SDN Controller

1. In this activity, we will install the RYU controller.

2. Ryu is a component-based SDN framework. It provides software components with well-defined APIs that enables developers to create new network management and control applications ^.

3. Ryu supports various SDN protocols, including OpenFlow. Ryu's functionalities and features can be explored on GitHub *.

4. In this activity, we will start by downloading and installing the Ryu controller which will be used as a remote controller in our further lab tasks.

5. To download and get started with the Ryu controller, simply issue the command "`pip3 install ryu`" at your home directory location.

6. Besides, we need to use "curl" to get data from a server. Therefore, download "curl" in your terminal by type this in your terminal

```
sudo apt update
sudo apt upgrade
sudo apt install curl
```

Task 3: Invoke the Ryu controller as a simple switch using a stock component.

1. In this activity, we will use the built-in component and start Ryu as a simple switch.

2. To implement the switching hub, we will use the component named "`ryu_manager.app.simple_switch_13.py`". The code for this component can be found in GitHub*.

3. This code will enable a networking device to function as a simple switch, flooding any packet it receives initially. It then associates the port to the destination MAC address

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

of the host, remembering this information. This information will then be used next time the device receives a packet, to avoid flooding.

4. We will start by issuing the command to bring up the Ryu controller using the stock component for the simple switch
5. "`ryu-manager ryu.app.simple_switch_13`".

6. We will then create a topology in mininet by issuing the command "`sudo mn --switch ovsk --controller remote --custom ceg_topo.py --topo cegTopo`". The "*ceg_topo.py*" file creates a simple topology with one switch and 5 hosts connected to it.

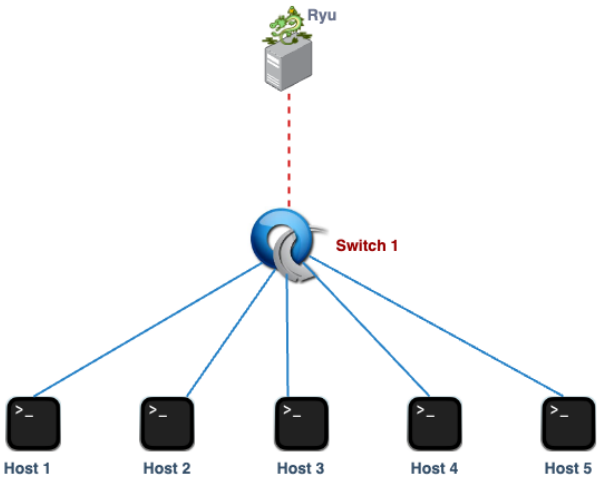


Figure 1: Topology

```
(base) divya@divya:~/mininet/examples$ sudo mn --switch ovsk --controller remo
te --custom ceg_topo.py --topo cegTopo
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1
*** Adding links:
(40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (s1, h1) (40.00Mbit 15ms delay)
(40.00Mbit 15ms delay) (s1, h2) (40.00Mbit 15ms delay) (40.00Mbit 15ms delay)
(s1, h3) (40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (s1, h4) (40.00Mbit 15m
s delay) (40.00Mbit 15ms delay) (s1, h5)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 1 switches
s1 ...(40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (40
.00Mbit 15ms delay) (40.00Mbit 15ms delay)
*** Starting CLI:
mininet-wifi>
```

```
(base) divya@divya:~/mininet/examples$ ryu-manager ryu.app.simple_switch_13
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
packet in 1 da:45:14:9b:4d:73 33:33:00:00:00:16 4
packet in 1 72:65:aa:d7:10:64 33:33:ff:d7:10:64 3
packet in 1 4a:ba:b5:ec:94:3f 33:33:00:00:00:16 2
packet in 1 22:b4:bf:ab:d0:eb 33:33:ff:ab:d0:eb 1
packet in 1 aa:1a:e1:12:cc:f4 33:33:ff:12:cc:f4 5
packet in 1 da:45:14:9b:4d:73 33:33:00:00:00:02 4
packet in 1 da:45:14:9b:4d:73 33:33:00:00:00:16 4
packet in 1 4a:ba:b5:ec:94:3f 33:33:00:00:00:02 2
packet in 1 4a:ba:b5:ec:94:3f 33:33:00:00:00:16 2
packet in 1 da:45:14:9b:4d:73 33:33:00:00:00:16 4
packet in 1 22:b4:bf:ab:d0:eb 33:33:00:00:00:02 1
packet in 1 22:b4:bf:ab:d0:eb 33:33:00:00:00:16 1
packet in 1 72:65:aa:d7:10:64 33:33:00:00:00:16 3
packet in 1 72:65:aa:d7:10:64 33:33:00:00:00:02 3
packet in 1 72:65:aa:d7:10:64 33:33:00:00:00:16 3
packet in 1 aa:1a:e1:12:cc:f4 33:33:00:00:00:16 5
packet in 1 aa:1a:e1:12:cc:f4 33:33:00:00:00:02 5
packet in 1 22:b4:bf:ab:d0:eb 33:33:00:00:00:16 1
packet in 1 aa:1a:e1:12:cc:f4 33:33:00:00:00:16 5
packet in 1 4a:ba:b5:ec:94:3f 33:33:00:00:00:16 2
```

7. Next, we shall check the flows in switch s1 in a terminal – We see that the packets will be sent to the controller.

```
(base) divya@divya:~/mininet$ sudo ovs-ofctl dump-flows s1
 cookie=0x0, duration=470.458s, table=0, n_packets=50, n_bytes=3788, priority=0 actions=CONTROLLER:65535
```

8. We now send a ping command from h1 to h2 in the mininet CLI

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

```
mininet-wifi> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=132 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=60.7 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=60.4 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=60.4 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=60.3 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=60.3 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=60.3 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=60.3 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=60.2 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=60.2 ms
^C
--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 60.224/67.462/131.572/21.370 ms
```

9. Next, we print the flows again, and notice that the controller has learnt the ports and associated the MAC addresses -

```
(base) divya@divya:~/mininet$ sudo ovs-ofctl dump-flows s1
 cookie=0x0, duration=220.946s, table=0, n_packets=11, n_bytes=1022, priority=1,i
n_port="s1-eth2",dl_src=4a:ba:b5:ec:94:3f,dl_dst=22:b4:bf:ab:d0:eb actions=output
:"s1-eth1"
 cookie=0x0, duration=220.912s, table=0, n_packets=10, n_bytes=924, priority=1,in
_port="s1-eth1",dl_src=22:b4:bf:ab:d0:eb,dl_dst=4a:ba:b5:ec:94:3f actions=output:
"s1-eth2"
 cookie=0x0, duration=1013.805s, table=0, n_packets=59, n_bytes=4390, priority=0
actions=CONTROLLER:65535
```

Task 4: To develop and test the code for a simple switching hub using Ryu's APIs.

4.1 Ryu's APIs can be used to create network management applications. Here, we provide the code for a simple switching hub in "`hub_controller.py`". A hub typically floods packets to all ports.

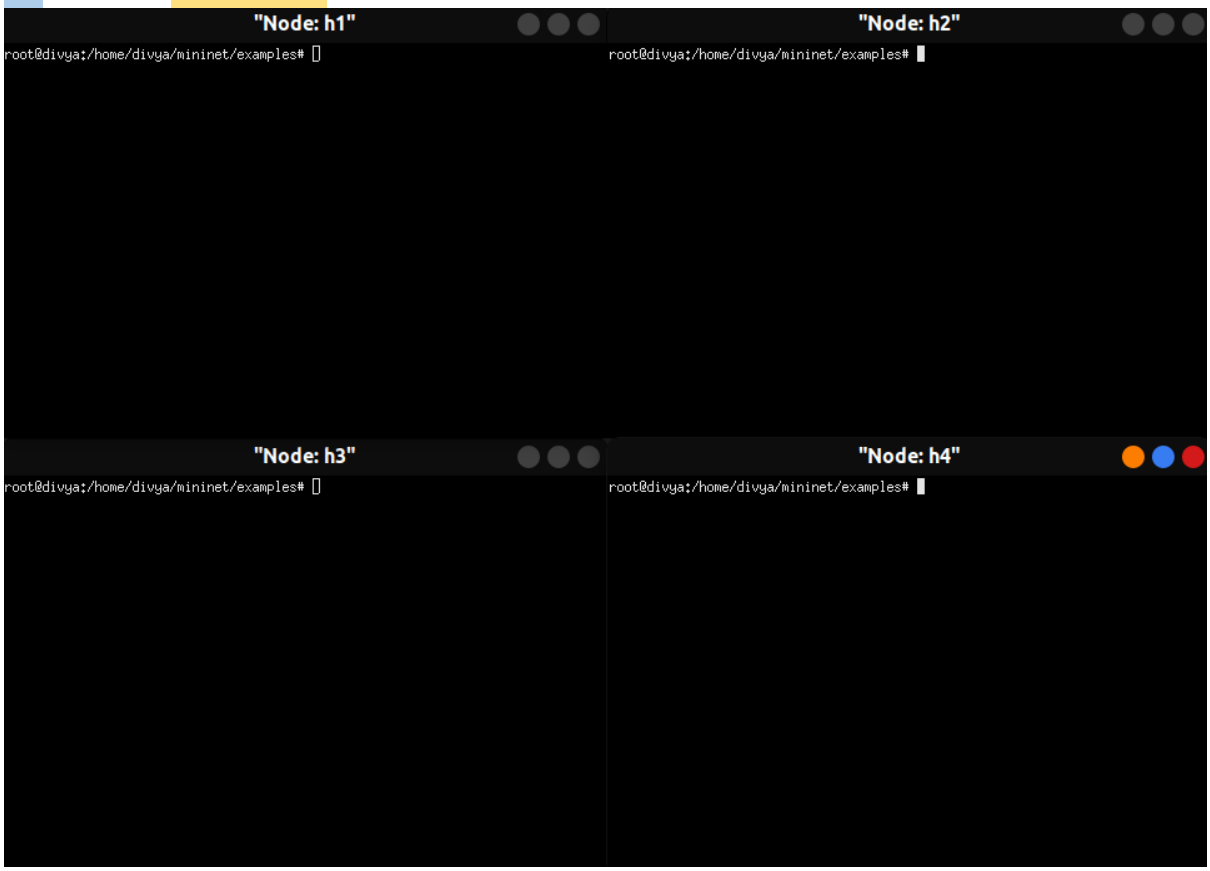
4.2 The Python code can be run in a similar manner with the command "`ryu-manager /path/to/hub_controller.py`". And we run the previous topology ""`sudo mn --switch ovsk --controller remote --custom ceg_topo.py --topo cegTopo`".

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

```
(base) divya@divya:~/mininet/examples$ sudo mn --switch ovsk --controller remot
e --custom ceg_topo.py --topo cegTopo
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1
*** Adding links:
(40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (s1, h1) (40.00Mbit 15ms delay) (
40.00Mbit 15ms delay) (s1, h2) (40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (s
1, h3) (40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (s1, h4) (40.00Mbit 15ms d
elay) (40.00Mbit 15ms delay) (s1, h5)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 1 switches
s1 ...(40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (40.00Mbit 15ms delay) (40.
00Mbit 15ms delay) (40.00Mbit 15ms delay)
*** Starting CLI:
mininet-wifi>
```

```
(base) divya@divya:~/mininet/examples$ ryu-manager hub_controller.py
loading app hub_controller.py
loading app ryu.controller.ofp_handler
instantiating app hub_controller.py of HubController
instantiating app ryu.controller.ofp_handler of OFPHandler
Handshake done with network device 0000000000000001
Flow-Mod added to 0000000000000001
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
Flooding packet
```

4.3 Next, open terminals for all four hosts by issuing the command "`xterm h1 h2 h3 h4`" over the Mininet CLI.

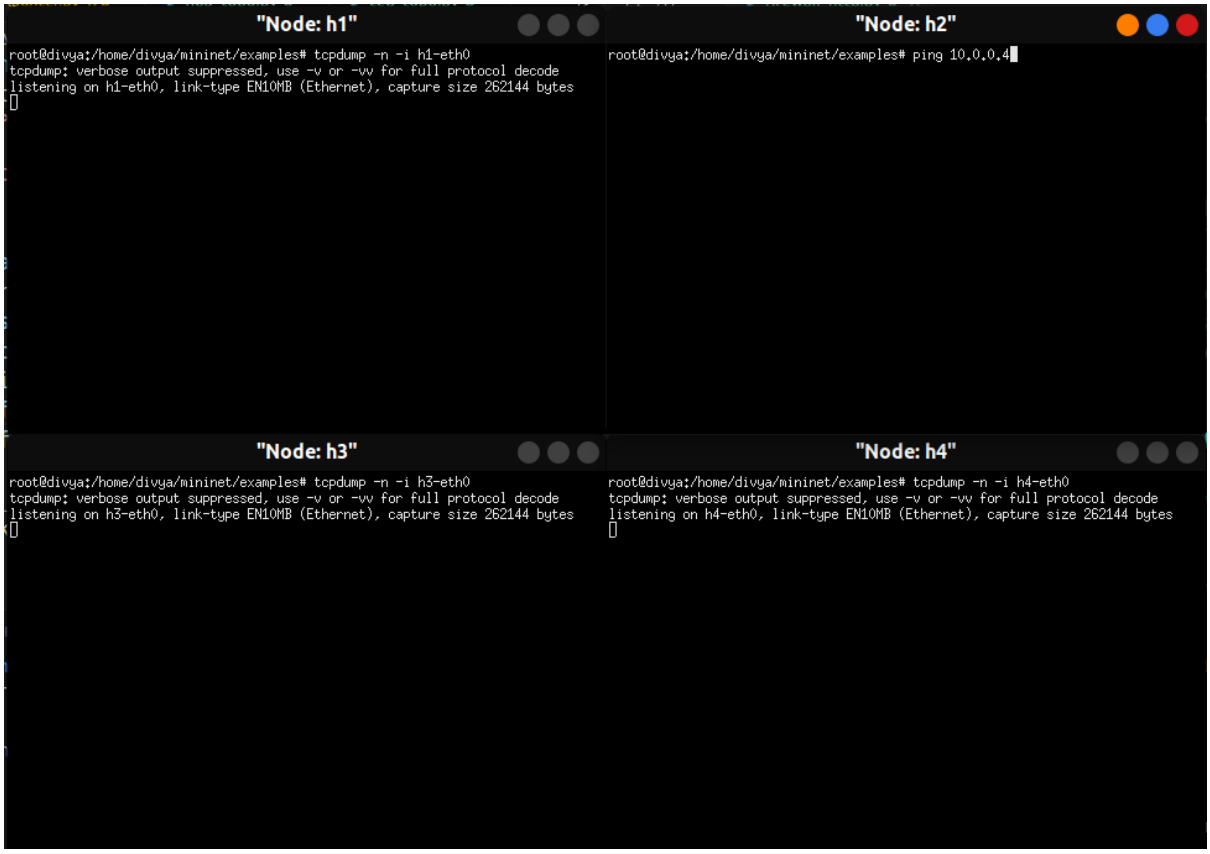


4.4 To verify the successful functioning, we will start sending packets from one of the hosts and monitor the flow of traffic on all connected hosts.

4.6 Next, issue the command "`tcpdump -n -i hX-eth0`" for all hosts except one because that would be used to send the ping command. Here, X is the number of the host.

4.7 In this activity, we issue tcpdump over h1, h3, and h4, and we use h2 to send a ping command toward h4.

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu



4.8 Once the ping starts, we will notice that since at first switch doesn't have any information regarding h4 therefore the first packet is sent to all hosts as per the layer-2 forwarding rule. However, only h4 will send a reply. NOTE: at h1 and h3 you can notice a line within TCP traffic says "Request who-has 10.0.0.4 tell 10.0.0.2, length 28".

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu



Task 5: To learn to test using Curl and HTTP Requests

5.1 We will use the "*ceg_topo.py*" topology network and start the HTTP server at h1 and h2. To do so, open the h1 and h2 terminals and start a server at h1 with port number 80 by issuing the command "`python3 -m http.server 80`".

5.2 At h2, we will use the Linux command "`curl`" to send the traffic to the server at its IP address "10.0.0.1".



^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

5.3 Then we observe h1 responding to the request (via the controller) -



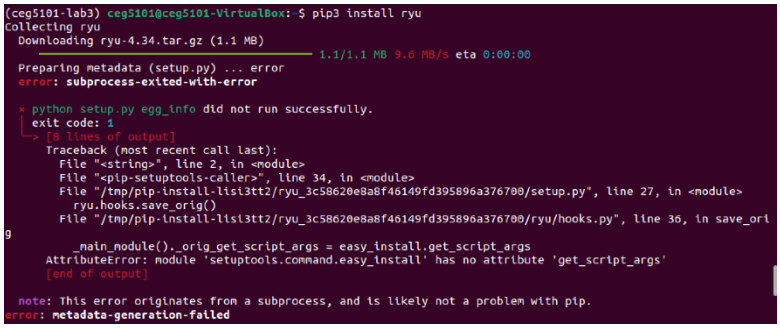
^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

Installing Ryu

Notes: There is an issue if your VM's Python version is >= 3.10 (e.g., Ubuntu 22.04). Ryu is not fully compatible with this version, so some adjustments are needed. One possible solution is to build a virtual environment with elder version of Python.

First, download and create a Conda virtual environment with Python 3.8.

1. Download the Miniconda installer
   a) wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
2. Run the installer
   a) bash Miniconda3-latest-Linux-x86_64.sh
3. Activate conda
   a) source ~/.bashrc
4. Test installation
   a) conda –version
5. Build a conda environment with python 3.8, named “myenv”
   a) conda create --name myenv python=3.8
   b) conda activate myenv
6. Install ryu
   a) Pip3 install ryu
7. Possible bugs
   a) Incompatibility due to the setuptools removing some functions that Ryu relies on.
   b) Try to fix by running
      i. Pip3 install setuptools==57.5.0
      ii. Pip3 install ryu



Another bug may appear when you test “ryu-manager”:

```
(ceg5101-lab3) ceg5101@ceg5101-VirtualBox:~$ ryu-manager
Traceback (most recent call last):
  File "/home/ceg5101/miniconda3/envs/ceg5101-lab3/bin/ryu-manager", line 5, in <module>
    from ryu.cmd.manager import main
  File "/home/ceg5101/miniconda3/envs/ceg5101-lab3/lib/python3.8/site-packages/ryu/cmd/manager.py", line 33, in <module>
    from ryu.app import wsgi
  File "/home/ceg5101/miniconda3/envs/ceg5101-lab3/lib/python3.8/site-packages/ryu/app/wsgi.py", line 109, in <module>
    class _AlreadyHandledResponse(Response):
  File "/home/ceg5101/miniconda3/envs/ceg5101-lab3/lib/python3.8/site-packages/ryu/app/wsgi.py", line 111, in _AlreadyHandledResponse
    from eventlet.wsgi import ALREADY_HANDLED
ImportError: cannot import name 'ALREADY_HANDLED' from 'eventlet.wsgi' (/home/ceg5101/miniconda3/envs/ceg5101-lab3/lib/python3.8/site-pa
ckages/eventlet/wsgi.py)
```

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu

It is due to the new version of "eventlet" having removed some functions. Choose the proper version of eventlet.
Follow the steps below to solve it:

1. Uninstall the current eventlet:
    - Pip3 uninstall eventlet
2. Install the proper version:
    - Pip3 install eventlet==0.30.2

After that, it should work.

^https://ryu.readthedocs.io/en/latest/getting_started.html
*https://github.com/faucetsdn/ryu